

InkCloser: Line-Art Completion for Paint-Bucket Colorization

Author Name

Affiliation

email@example.org

Abstract

Flood-fill (paint-bucket) coloring of line art requires the line drawing to form closed regions. In practice, line art produced by drawing, scanning, or automatic line extraction often contains small gaps, and a single gap lets the fill leak across the intended boundary. We address the problem of restoring such gaps so that a standard flood-fill tool can be applied without manual retouching. Unlike conventional image inpainting, the location of the missing content is not available at inference time. We present InkCloser, a U-Net with gated convolutions, a dilated context stack, and transformer self-attention at two resolutions. We train it on pairs built by synthetically removing segments of ink from a collection of clean, closed line art images, with a loss that combines pixel-wise cross-entropy, a global Dice term, and a Dice term restricted to the neighborhood of the removed segments. We also introduce evaluation metrics that measure gap closing and paint-bucket behavior directly, rather than relying on pixel accuracy. On a held-out test the model closes 70.5% of the gaps, raises the fraction of regions that a single bucket click fills correctly (bucket accuracy) from 64.1% to 94.7%, reduces the number of fill leaks per image from 15.5 to 1.5, and makes 45.0% of the drawings entirely leak-free.

1 Introduction

Two-dimensional cartoon and animation production commonly works with line art: dark strokes on a light background that delimit the regions of a drawing. Such drawings are typically colored with a flood-fill, or paint-bucket, tool, which fills an enclosed region in one click but requires closed contours: where a stroke is interrupted, the fill leaks into the neighboring region. Gaps are common, whether from drawing, scanning, compression, line thinning, or strokes that do not fully close a shape. Most drawing software mitigates this with a gap-closing option that treats openings below a user-set size as closed. This handles many small gaps, but the threshold involves a trade-off: low values let wider gaps leak, while high values also close narrow passages between nearby strokes.

Our goal is to repair the line art itself: given a drawing with interrupted strokes, we restore the missing ink so that it supports flood-fill coloring without manual correction or per-drawing tuning. This complements the gap-closing options of fill tools, handling gaps beyond their reach and producing line art that is closed for any tool or later pipeline step.

The problem is similar to image inpainting, with the key difference that the regions to be filled are not known at inference time. The model must implicitly locate the gaps and close them, while leaving the rest of the drawing unchanged, which makes this a blind, or mask-free, restoration task. A particular difficulty is that not every discontinuity in a line drawing is a defect. Line art routinely contains strokes that end freely by design (e.g. interior detail lines such as folds, wrinkles, etc.). Whether an opening is a gap to be closed or an intentional stroke ending cannot be decided from local geometry alone and can only be learned from examples of complete, correctly closed line art.

To obtain such examples, we collected images from repositories of coloring pages, which are made specifically to be filled region by region. We generated corrupted inputs by synthetically

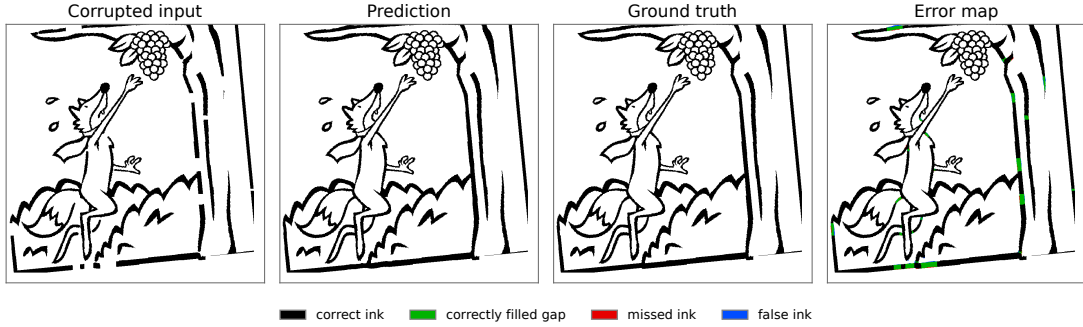


Figure 1: Line-art restoration example. From left to right: corrupted input with missing stroke segments, binarized prediction of our model, ground truth, and error map.

erasing segments of their ink. We use the resulting dataset to train a network that combines local convolutional processing with global attention, so that it can bridge gaps much longer than a local filter’s reach. Because the erased strokes are a tiny fraction of the image, we design the loss function so that the training signal concentrates on them. The contributions of this work are:

- A network architecture for mask-free gap closing in line art, which locates and bridges gaps without a hole mask, with the goal of enabling flood-fill coloring.
- A loss function that combines hole-weighted cross-entropy with global and gap-local Dice terms, focusing training on the erased strokes while preserving the intact drawing.
- A set of evaluation metrics tailored to flood-fill coloring rather than to pixel-wise reconstruction, measuring gap closure per erased segment and simulating the paint-bucket tool through bucket accuracy and fill leaks.
- A public dataset of 55 000 corrupted and clean line art pairs [Ruben Tous, 2026].

2 Related Work

Image inpainting. Restoring missing strokes resembles image inpainting, in which a network fills a masked region of an image from its surrounding context. Learned inpainting methods handle irregular holes by conditioning each convolution on which input pixels are valid [Liu et al., 2018, Yu et al., 2019], and more recent models enlarge the receptive field to fill large holes, for example with fast Fourier convolutions [Suvorov et al., 2022]. These methods assume that a mask of the corrupted region is given, both during training and at test time. Blind inpainting removes this assumption: VCNet [Wang et al., 2020] first predicts which pixels are corrupted and then inpaints them, but it targets natural images, where corruption appears as visible foreign patterns. In line art, a gap is simply missing ink and looks like background. In our setting the mask is used only to build training pairs: at inference time the network must also find the gaps it fills.

Gap handling at coloring time. The gap problem is well known in cartoon coloring, and the classic remedies avoid leaks without any learning. Morphological closing of the ink mask [Serra, 1982, Soille, 2003], a dilation followed by an erosion, bridges gaps up to the size of the structuring element, but it also fills narrow background areas and sharp corners. Trapped-ball segmentation [Zhang et al., 2009] floods the background with a ball that cannot pass through gaps narrower than its radius, and LazyBrush [Sýkora et al., 2009] formulates scribble-based coloring as an energy minimization that tolerates small gaps in the line art. Fourey et al. [2018] detect stroke

endpoints from contour curvature and close gaps with splines and straight segments, subject to geometric constraints. Their algorithm is the basis of the line-art detection mode of the bucket fill tool in GIMP. Delaunay Painting [Parakkat et al., 2022] builds a Delaunay triangulation of the contours and propagates user color hints over it, which makes coloring robust to large gaps. These methods prevent leaks at coloring time, but they either do not restore the missing strokes or close them with hand-designed geometric rules and size thresholds. Recent work also addresses the opposite problem of small regions left *unpainted* after bucket filling: GapFill [Kono et al., 2026] detects such regions and suggests their colors. This shows that the paint-bucket workflow is still an active research subject, and the problem it solves is complementary to ours.

Learned line-art restoration and cleanup. Learned approaches instead repair the line art itself. The closest to ours is the work of Sasaki et al. [2017], who train a fully convolutional encoder-decoder without skip connections on synthetically corrupted line drawings, with a mean squared error loss, so that the network jointly detects and inpaints gaps without a mask. The same authors later extended the idea to the restoration of deteriorated historical drawings [Sasaki et al., 2018]. Related cleanup tasks include sketch simplification, which turns rough sketches into clean lines and has been formulated with explicit attention to region closure [Liu et al., 2015] and learned with adversarial augmentation [Simo-Serra et al., 2018], and sketch completion on vector stroke sequences [Lin et al., 2020]. These works aim at the visual completion or cleanliness of the drawing, whereas our objective is flood-fill coloring: we evaluate whether regions are correctly closed, and we design the architecture and the loss around that goal. We compare with Sasaki et al. [2017] directly in Section 7.

Line art in animation pipelines. Several recent learning-based tools for 2D animation operate on the regions or strokes of clean line art and therefore depend on it being well formed. Reference-based colorization methods match the segments enclosed by the lines across frames [Casey et al., 2021]. Dai et al. [2024b] formulate paint-bucket colorization as an inclusion-matching problem between segments and introduce the PaintBucket-Character dataset, and extend the approach to coloring from character design sheets [Dai et al., 2024a]. Because these methods assign one color per enclosed region, a single gap merges two regions and propagates the error to both. Line inbetweening [Siyao et al., 2023] and joint stroke tracing and correspondence [Mo et al., 2024] likewise build on vertex or stroke representations of clean line drawings. Diffusion-based colorization [Liu et al., 2025] does not require closed regions, since it generates pixel colors directly, but it does not produce the flat, region-wise fills of the paint-bucket workflow. Our method is a preprocessing step for this family of region-based tools: it supplies the closed contours that they assume.

Topology-aware losses. Closing a gap is a topological rather than a pixel-wise property: a few missing pixels can merge two regions while barely changing overlap-based losses. In the segmentation of thin structures such as vessels and roads, this has motivated losses that explicitly preserve connectivity, using persistent homology [Hu et al., 2019] or a centerline-based Dice term (cDice) [Shit et al., 2021]. Our gap Dice term follows a related intuition with a simpler mechanism: it concentrates the overlap objective on the neighborhood of the removed segments, where connectivity is decided.

3 Problem Statement

Let $Y \in \{0, 1\}^{H \times W}$ be a clean line art image (0 for ink, 1 for background), and let $X \geq Y$ be a corrupted version in which a subset of ink pixels, the gap or hole region, has been turned into background. Recovering Y amounts to redrawing the ink in the hole region while leaving the rest of X unchanged.

Mask-guided inpainting would learn a mapping that receives both X and the hole region, so that only the content of the holes must be predicted. We instead learn a mapping f_θ with $f_\theta(X) \approx Y$, as in Sasaki et al. [2017]: the hole region follows from the pair (X, Y) and is used only during training, so at inference the network must also locate it, which requires distinguishing interrupted strokes from strokes that end by design.

4 Dataset

The InkCloser Dataset is built from 10 000 clean, closed monochrome line art images collected from coloring-page datasets such as the Coloring Book Objects Dataset [Chen et al., 2019]. Each image is resized to 896×896 pixels, and corrupted versions are generated by synthetically removing segments of ink. Corruption is applied in two stages, both operating on the skeleton of the strokes. The first stage targets T-shaped junctions, where one line meets another: it erases the first few pixels of the arriving line while keeping the other line intact. This reproduces a common defect of hand-drawn line art, in which a stroke stops just short of the line it should touch, and is applied to between 50% and 100% of the eligible junctions of each image. The second stage cuts gaps in the middle of strokes. To keep the drawings recognizable, at most 35% of any stroke segment and 25% of any connected shape is removed, and small shapes such as eyes or highlights are never corrupted.

The base images are split into 9 000 for training and 1 000 for testing. Each training image has 6 corrupted variants (54 000 pairs), and each test image has a single fixed corrupted version (1 000 pairs). The resulting 55 000 pairs form the InkCloser Dataset, which we make public for reproducibility [Ruben Tous, 2026].

During training, each time an image is drawn, one of its variants is selected at random, so the network sees different gap patterns for the same clean image without corrupting images on the fly. The test set uses a single frozen variant per image, so that test metrics are reproducible and every method compared in Section 7 sees exactly the same gaps.

Training pairs are also augmented. Corruption is applied to the clean image before augmentation, so that gap boundaries remain sharp and are not softened by resampling. The corrupted and clean images are then stacked and passed through the same random horizontal flip ($p = 0.5$), vertical flip ($p = 0.2$), rotation (up to 10°), and affine translation and scaling ($\pm 5\%$), so that they remain pixel-aligned; regions introduced by rotation or translation are filled with white.

5 Method

5.1 Architecture

The network, illustrated in Figure 2, is an encoder-decoder with skip connections, operating on single-channel 896×896 images. Every convolution in the encoder is a gated convolution [Yu et al., 2019]: it computes a feature map and, in parallel, a sigmoid gate from the same input, and multiplies the two element-wise. This lets the network learn, at every layer and spatial location, how much weight to give to a region of the input, which is useful here because parts of the input are unreliable by construction (the gaps).

The encoder has six stages. The first stage operates at full resolution (896×896) without downsampling; the following five stages each halve the spatial resolution with a strided gated convolution, down to 28×28 , doubling the number of channels at each step (16, 32, 64, 128, 256, 512). Each stage is followed by Group Normalization [Wu and He, 2018] and a ReLU activation. The decoder mirrors the encoder with transposed convolutions for upsampling and concatenates the corresponding encoder feature map at each resolution before a regular convolution, following

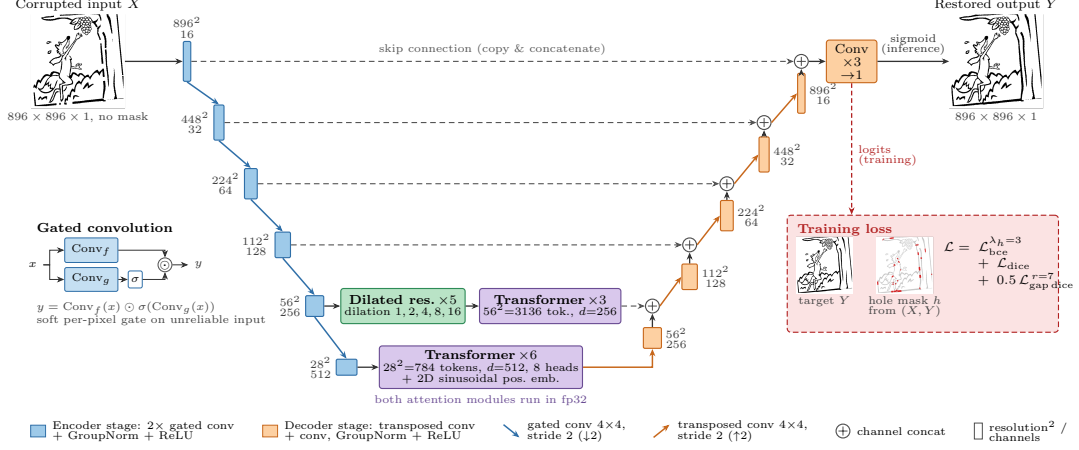


Figure 2: Overview of the restoration pipeline: a corrupted line art image is passed through the network with no mask, and the network outputs a restored image with the gaps closed. Placeholder figure.

the standard U-Net skip-connection pattern [Ronneberger et al., 2015]. The final stage produces a single-channel logit map at the input resolution.

Two components are added to the plain U-Net to help the network reason about context beyond the receptive field of a stack of 3×3 convolutions, which is necessary when a gap is large relative to the stroke width.

- At the 28×28 bottleneck, a stack of six transformer blocks [Vaswani et al., 2017] with 8 attention heads operates on the flattened spatial positions as a token sequence, with a fixed 2D sinusoidal positional embedding added to each token. This lets any position in the feature map attend to any other position, which is useful for propagating information about line continuation across an arbitrarily large gap.
- At the 56×56 resolution, a stack of five dilated residual blocks with dilation rates 1, 2, 4, 8, 16 is applied first, followed by a smaller stack of three transformer blocks with the same attention design. The dilated blocks expand the receptive field cheaply before the more expensive attention operation is applied at this larger resolution.

Both attention modules are computed in single precision regardless of the mixed precision setting used elsewhere in the network, which we found necessary for numerically stable training.

The network outputs logits during training, so that the loss can use a numerically stable binary cross-entropy formulation; a sigmoid is applied only when the network is in evaluation mode, turning the output into a per-pixel probability. The full model has 43.4 M parameters.

5.2 Loss function

Two properties of the task motivate the design of the loss. First, ink pixels are a small minority of all pixels, so an unweighted pixel-wise loss is dominated by the background and gives a weak training signal for the gap-closing behavior we care about. Second, we want the loss to specifically reward closing the gaps, not only to reward reproducing the overall image, since a network that copies the input unchanged already achieves a low loss almost everywhere except at the gaps.

Let $p_{ij} = \sigma(\hat{Y}_{ij})$ denote the predicted probability of background at pixel (i, j) from the network’s logits, and let $h_{ij} \in \{0, 1\}$ denote the hole indicator defined in Section 4. The loss combines three terms.

Hole-weighted cross-entropy. A pixel-wise binary cross-entropy term is computed against the clean target and reweighted so that hole pixels count more than non-hole pixels:

$$L_{\text{bce}} = \frac{1}{HW} \sum_{i,j} (1 + h_{ij}(\lambda_h - 1)) \text{BCE}(\hat{Y}_{ij}, Y_{ij}), \quad (1)$$

with hole weight $\lambda_h = 3$.

Global Dice. A Dice loss [Milletari et al., 2016] is computed between the predicted ink probability map, $1 - p_{ij}$, and the binarized ink target over the whole image, which further counters the class imbalance between ink and background:

$$L_{\text{dice}} = 1 - \frac{2 \sum_{i,j} (1 - p_{ij}) \mathbb{I}[Y_{ij} < 0.5] + \epsilon}{\sum_{i,j} (1 - p_{ij}) + \sum_{i,j} \mathbb{I}[Y_{ij} < 0.5] + \epsilon}. \quad (2)$$

Gap Dice. The same Dice term is computed a second time, restricted to a neighborhood of the hole region obtained by dilating the hole mask with a $(2r + 1) \times (2r + 1)$ max-pooling window, $r = 7$ pixels. This term is zero for images without gaps and otherwise focuses the Dice objective specifically on the completed regions, independently of how well the rest of the image is reconstructed.

The total loss is

$$L = \beta_{\text{bce}} L_{\text{bce}} + \beta_{\text{dice}} L_{\text{dice}} + \beta_{\text{gap}} L_{\text{gap dice}}, \quad (3)$$

with $\beta_{\text{bce}} = 1.0$, $\beta_{\text{dice}} = 1.0$, and $\beta_{\text{gap}} = 0.5$. In preliminary experiments, a much larger hole weight ($\lambda_h = 8$) caused the network to over-predict ink outside the hole regions; the milder weight of 3, combined with the Dice terms, gave a better balance between filling gaps and preserving the rest of the drawing.

5.3 Training procedure

The network is trained with AdamW [Loshchilov and Hutter, 2019] and weight decay 10^{-4} , using two learning rates: 10^{-3} for all convolutional parameters and 3×10^{-4} for the parameters of the two attention modules. The learning rate is decayed with a cosine annealing schedule [Loshchilov and Hutter, 2017] down to a minimum of 10^{-6} . Training uses mixed-precision forward and backward passes with gradient scaling, and gradients are clipped to a maximum norm of 1.0. The batch size is 4, giving 2 250 optimization steps per pass over the 9 000 training images; at each step, one of the pre-generated corrupted variants is drawn at random for every image in the batch.

6 Evaluation Protocol and Metrics

Pixel-wise accuracy or a plain pixel F1 score is a poor proxy for the quality of the restoration in this task, for two reasons: ink pixels are a small fraction of the image, so a model that never adds any ink already scores well on pixel accuracy; and pixel overlap does not directly say whether the regions that flood fill depends on were actually recovered. We therefore report metrics at three levels: the ink added by a method, the individual erased segments, and the background regions that a paint-bucket tool would fill. Global pixel metrics are reported for reference.

All metrics are computed on the binarized output. A pixel is ink if its value is below 0.5 on a $[0, 1]$ scale, and the network’s probability map is thresholded at 0.5. A region is a 4-connected component of background pixels, which pairs correctly with 8-connected ink strokes; background components smaller than 20 pixels are ignored, since they are anti-aliasing specks rather than paintable regions. Unless noted otherwise, metrics are pooled over the whole test set (micro-averaged). Counts are summed over all images and the ratio is taken once.

Fill precision, recall, and F1. These metrics consider only the ink that a method adds. Let added be the set of pixels that are ink in the output but background in the corrupted input, and let gap be the set of pixels that are ink in the clean target but background in the corrupted input (the ground-truth hole region). With a pixel tolerance τ (a match within τ pixels, via morphological dilation, counts as correct), fill precision is the fraction of added pixels that lie within τ pixels of the ground-truth ink, and fill recall is the fraction of gap pixels that lie within τ pixels of the added ink. Fill F1 is their harmonic mean. We report $\tau = 0$ (exact) and $\tau = 2$. The exact setting is strict: a correctly placed stroke that is one pixel thicker than the target is penalized. The tolerant setting separates errors in line thickness or position from missing structure.

Pixel metrics. Pixel IoU and pixel F1 between the output ink and the ground-truth ink over the whole image; *ink kept*, the fraction of the input’s ink pixels that remain ink in the output, which checks that a method does not erase existing lines; and the *ink ratio*, the total output ink divided by the total ground-truth ink, which is 1 in the ideal case, above 1 when lines are too thick or spurious ink is added, and below 1 when ink is missing.

Gap closure. An erased segment is an 8-connected component of the ground-truth hole region, ignoring components of fewer than 4 pixels. A segment is counted as closed if at least 80% of its pixels are ink in the output. Gap closure is the fraction of erased segments that are closed. The length of a segment is the longer side of its bounding box, in pixels, which we use to analyze performance as a function of gap size.

Region and paint-bucket metrics. These metrics measure the practical goal of the task most directly. *Region count accuracy* is the fraction of images in which the output has exactly as many regions as the ground truth; it is a strict, whole-image metric, since a single missed or spurious gap changes the count. *Region recovery* is the fraction of regions merged by the gaps (ground-truth region count minus input region count, summed over images) that are separated again in the output. To simulate the paint-bucket tool, each ground-truth region is matched to the output region with the highest IoU with it; the region is *correctly paintable* if that IoU is at least 0.9. *Bucket accuracy* is the fraction of ground-truth regions that are correctly paintable, which approximates the fraction of regions that a user could fill correctly with a single click, and *mean region IoU* is the average best-match IoU over all ground-truth regions. Finally, a *leak* occurs when one output region contains at least 50% of each of two or more ground-truth regions, so that a single bucket fill would spill across them; an output region that absorbs k ground-truth regions counts as $k - 1$ leaks. We report the mean number of *leaks per image* and the fraction of *leak-free images*, that is, drawings that are fully ready to color without manual correction.

Statistical analysis. We report 95% percentile bootstrap confidence intervals [Efron and Tibshirani, 1994] computed over test images (1000 resamples). The same resamples are used for all methods, so differences between methods are paired, and we report one-sided paired bootstrap tests of the proposed method against each baseline.

Baselines. As no mask is available at inference time, mask-guided inpainting methods are not directly applicable. We compare against the following methods, all of which operate under the same mask-free, single-image setting and are evaluated on exactly the same test pairs with the same metric code:

- *Input (no restoration)*: the corrupted image evaluated as if it were the output. It is the lower bound and quantifies the damage caused by the gaps. Since it adds no ink, fill precision and fill F1 are undefined for it.

Table 1: Line restoration on the held-out test set (1 000 images, 26 141 erased segments). Values are percentages unless noted; $\pm$ is half the width of the 95% bootstrap confidence interval. τ : pixel tolerance. Bucket acc.: ground-truth regions recovered with $\text{IoU} \geq 0.9$. Leak: one output region spanning two ground-truth regions. $\uparrow$: higher is better; $\downarrow$: lower is better; $\rightarrow 1$: closest to 1 is best. Best value per column in bold.

Method	Added ink				Whole image		Segments	Paint bucket		
	Fill P ($\tau=0$) $\uparrow$	Fill R ($\tau=0$) $\uparrow$	Fill F1 ($\tau=0$) $\uparrow$	Fill F1 ($\tau=2$) $\uparrow$	Pixel F1 $\uparrow$	Ink ratio $\rightarrow 1$	Gap closure $\uparrow$	Bucket acc. $\uparrow$	Leaks / image $\downarrow$	Leak-free images $\uparrow$
Input (no restoration)	—	0.0 $\pm$ 0.0	—	—	98.4 $\pm$ 0.0	0.968 $\pm$ 0.001	0.0 $\pm$ 0.0	64.1 $\pm$ 1.6	15.46 $\pm$ 0.66	0.1 $\pm$ 0.1
Closing ($k=5$)	1.1 $\pm$ 0.1	1.8 $\pm$ 0.1	1.4 $\pm$ 0.1	11.6 $\pm$ 0.4	95.9 $\pm$ 0.1	1.021 $\pm$ 0.003	1.4 $\pm$ 0.2	37.6 $\pm$ 1.1	13.08 $\pm$ 0.58	0.2 $\pm$ 0.2
Closing ($k=9$)	1.2 $\pm$ 0.1	6.3 $\pm$ 0.2	2.1 $\pm$ 0.1	25.2 $\pm$ 0.7	91.1 $\pm$ 0.4	1.129 $\pm$ 0.009	6.3 $\pm$ 0.4	22.8 $\pm$ 1.1	9.45 $\pm$ 0.43	0.3 $\pm$ 0.4
Sasaki et al. (pretrained)	10.7 $\pm$ 0.5	54.0 $\pm$ 1.0	17.8 $\pm$ 0.7	81.9 $\pm$ 0.6	90.1 $\pm$ 0.3	1.081 $\pm$ 0.003	26.6 $\pm$ 0.8	26.2 $\pm$ 1.5	13.77 $\pm$ 1.38	4.2 $\pm$ 1.2
Sasaki et al. (retrained)	28.1 $\pm$ 1.3	56.5 $\pm$ 1.3	37.6 $\pm$ 1.4	87.2 $\pm$ 0.7	94.1 $\pm$ 0.2	0.974 $\pm$ 0.001	20.8 $\pm$ 1.1	36.4 $\pm$ 2.1	19.36 $\pm$ 1.87	8.5 $\pm$ 1.6
Ours	73.6$\pm$1.0	84.0$\pm$0.6	78.5$\pm$0.7	92.8$\pm$0.5	99.3$\pm$0.0	1.004$\pm$0.000	70.5$\pm$0.9	94.7$\pm$0.4	1.47$\pm$0.14	45.0$\pm$3.0

- *Morphological closing* [Serra, 1982, Soille, 2003]: a dilation followed by an erosion of the ink mask with an elliptical structuring element of diameter k pixels, with $k \in \{5, 9\}$. It is the classical training-free way to close gaps, and is essentially what the “close gap” options of drawing software do. It can only bridge gaps shorter than about k pixels, and it also fills narrow background areas and sharp corners.
- *Sasaki et al.* [Sasaki et al., 2017]: the joint gap detection and inpainting network, a fully convolutional encoder-decoder without skip connections (5.5 M parameters). The official release targets Torch7; we converted the released weights layer by layer into an identical PyTorch network and followed the authors’ pre- and post-processing (normalization with the stored mean and standard deviation, white padding to a multiple of 16, and their output sharpening step, which does not change which pixels are ink at the 0.5 threshold). We evaluate two variants. *Pretrained*: the released weights applied zero-shot, with no training on our data. The only adaptation considered was the resolution at which the network runs; the scale factor was selected by fill F1 on 64 training images (never on test images), and the best choice was the native resolution (scale 1). *Retrained*: the same architecture trained from random initialization on our training pairs, with the same augmentation as our model, following the original training recipe (mean squared error loss and the AdaDelta optimizer [Zeiler, 2012]) on random 448×448 crops, and evaluated on the full test images. The last checkpoint of the run is reported, so no model selection was performed on test data. This variant isolates the effect of our architecture and training objective under identical data.

7 Results

All results are computed on the full frozen test set of 1 000 images and 26 141 erased segments. Every method is evaluated on the same corrupted images and with the same metric code, and the bootstrap confidence intervals of all methods use the same resamples. Table 1 reports the main metrics with their confidence intervals, and Table 4 in Appendix A lists every metric for every method.

Paint-bucket behavior. The metrics that measure the actual goal of the task show the largest effect. On the unrestored input, only 64.1% of the ground-truth regions can be filled correctly with a single bucket click, each image has on average 15.5 leaks, and only 1 of the 1 000 images is leak-free. After restoration with our model, bucket accuracy rises to 94.7%, the number of leaks per image drops by 90.5% to 1.47, and 45.0% of the drawings (450 images) are entirely leak-free and ready to color without any manual correction. The remaining leaks are concentrated in few images: 66.9% of the restored images have at most one leak and 78.5% have

at most two, compared with a median of 13 leaks per image on the input. Of the 15 458 regions that the gaps merge across the test set, 94.0% are separated again, and the mean best-match IoU of the ground-truth regions rises from 70.9% to 96.5%. Region count accuracy, the strictest whole-image metric, rises from 0.1% to 38.0%. Our model is the only method in Table 1 that improves on the bucket accuracy of the unrestored input; every other method lowers it.

Comparison with morphological closing. Morphological closing fails on this task. With $k = 9$ it closes only 6.3% of the erased segments and recovers 6.3% of the missing ink. Although it reduces the number of leaks per image (from 15.5 to 9.4), it does so largely by filling narrow background areas and thickening junctions, which destroys small regions rather than separating merged ones: its bucket accuracy drops to 22.8%, well below that of the unrestored input (64.1%), its ink ratio rises to 1.13, and its pixel F1 falls below that of the input. The smaller kernel ($k = 5$) damages the drawing less but closes almost no gaps (1.4%). Both effects are visible in the second column of Figures 5 and 7, where closing merges nearby strokes into ink blobs while leaving most gaps open. The fill precision of closing at $\tau = 2$ is high (100.0% for $k = 5$, 91.3% for $k = 9$) only because a dilation can add ink solely next to existing strokes; its recall at the same tolerance remains low (6.1% and 14.6%).

Comparison with Sasaki et al. The network of Sasaki et al. [2017] performs clearly better than closing at the ink level, but it does not make the drawings paintable. With its released weights, applied zero-shot, it redraws 54.0% of the erased ink and closes 26.6% of the erased segments, the highest gap closure among the baselines. However, only 10.7% of the ink it adds lies exactly on a ground-truth stroke, its ink ratio is 1.08, and it keeps only 95.1% of the ink of the input: the network redraws the whole drawing in the style of its own training data, with thicker strokes and small spurious marks, rather than only filling the gaps. As a consequence, it re-separates 60.6% of the merged regions but correctly recovers only 26.2% of the ground-truth regions, and only 4.2% of the images are leak-free. This variant was trained on different drawings, with different line widths and gap statistics, so these numbers measure zero-shot generalization rather than the capacity of the approach; changing the input scale did not improve its fill F1 on training images (Section 6).

Retrained on our training pairs, the same architecture adapts to the line style of our data: its fill precision rises to 28.1% (97.6% at $\tau = 2$), its fill F1 to 37.6%, and its ink ratio (0.974) is close to that of the input. Its gap closure, however, decreases to 20.8%, its region recovery to 33.7%, and the number of leaks per image increases to 19.4, above that of the unrestored input (15.5). We attribute this mainly to the fact that the network regenerates every stroke rather than copying the input where it is intact: it keeps only 94.1% of the input ink, and its error maps (Figure 5) show missed and false pixels along almost every existing stroke, not only at the gaps. Thin and faint strokes are partly lost in this process, which opens new gaps, and therefore new leaks, elsewhere in the drawing. Its results also vary strongly between images (Figure 3): its per-image bucket accuracy spans the whole range from 0 to 1, with a median of 48.3%, whereas ours has a median of 96.4%. Since this variant uses the same training data, augmentation, and test protocol as our model, the difference between the two, 49.7 percentage points in gap closure and 58.4 in bucket accuracy, reflects the combined effect of our architecture and training objective. The skip connections and gated convolutions let our model pass intact strokes through unchanged (it keeps 99.9998% of the input ink), while the attention modules and the hole-focused loss direct its capacity to the gaps.

Statistical significance. Our model outperforms the input, both closing variants, and both Sasaki et al. variants on all four key metrics (fill F1, gap closure, bucket accuracy, and leak-free images), with paired-bootstrap $p < 0.001$ in every comparison (Table 2). The per-image distributions in Figure 3 confirm that the improvement is not driven by a subset of easy images:

Table 2: Paired bootstrap comparison of our model against each baseline: difference (ours minus baseline, percentage points) with its 95% confidence interval, computed with the same 1 000 resamples of test images for all methods. All differences are significant with a one-sided $p < 0.001$ (the smallest reportable value with 1 000 resamples). Fill F1 is undefined for the input.

Baseline	Fill F1 ($\tau=0$)	Gap closure	Bucket acc.	Leak-free images
Input (no restoration)	–	+70.5 [69.5, 71.4]	+30.7 [29.3, 32.1]	+44.9 [41.9, 47.9]
Closing ($k=5$)	+77.1 [76.4, 77.8]	+69.1 [68.1, 70.0]	+57.2 [56.1, 58.3]	+44.8 [41.7, 47.8]
Closing ($k=9$)	+76.4 [75.7, 77.1]	+64.1 [63.1, 65.1]	+72.0 [70.8, 73.1]	+44.7 [41.6, 47.7]
Sasaki et al. (pretrained)	+60.7 [60.0, 61.3]	+43.9 [43.0, 44.8]	+68.5 [67.0, 70.1]	+40.8 [37.8, 43.7]
Sasaki et al. (retrained)	+40.9 [39.8, 42.1]	+49.7 [48.7, 50.6]	+58.4 [56.3, 60.5]	+36.5 [33.6, 39.4]

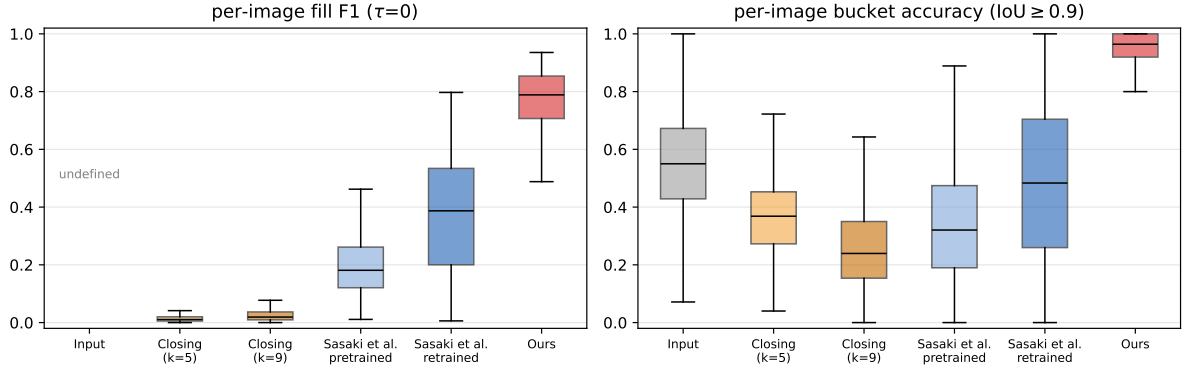


Figure 3: Per-image distributions over the 1 000 test images of the fill F1 at $\tau = 0$ (left) and of the bucket accuracy (right). Boxes show the interquartile range and the median; whiskers extend to 1.5 times the interquartile range; outliers are hidden. Fill F1 is undefined for the input.

the first quartile of the per-image bucket accuracy of our model (92.0%) is higher than the median of every other method, and higher than the upper whisker of both closing variants.

Ink-level accuracy. Our model recovers 84.0% of the erased ink exactly and places 73.6% of the ink it adds exactly on the ground-truth strokes, for a fill F1 of 78.5%. With a tolerance of two pixels, fill precision, recall, and F1 rise to 91.7%, 93.9%, and 92.8%, which indicates that most of the remaining pixel-level error is a matter of stroke thickness or position along the stroke boundary rather than missing or hallucinated structure. The model preserves the input: it keeps 99.9998% of the ink already present in the corrupted image, and its ink ratio (1.004) shows that the restored lines are neither systematically thicker nor thinner than the target. Pixel F1 and pixel IoU are high for all methods, including the unrestored input (98.4% pixel F1), because most ink is untouched by the corruption; they confirm that the model does not degrade the drawing (99.3% pixel F1) but are not informative for ranking methods. Every baseline, including both Sasaki et al. variants, has a lower pixel F1 than the unrestored input, because the ink it damages outweighs the ink it recovers.

Effect of gap length. Table 3 and Figure 4 break down gap closure by the length of the erased segment. Morphological closing only closes gaps that are short relative to its kernel: with $k = 9$ it closes 24.4% of the gaps shorter than 8 pixels, 3.3% of those between 8 and 16 pixels, and practically none above 16 pixels. The pretrained Sasaki et al. network closes between 23.8% and 31.8% of the gaps in each bin up to 64 pixels; retrained on our data, its closure rate grows

Table 3: Gap closure (%) by erased-segment length (longer side of its bounding box, in pixels). A gap is closed if at least 80% of its pixels are ink in the output. The input closes no gaps (0.0% in every bin) and is omitted.

Gap length (px)	# gaps	Closing		Sasaki et al.		Ours
		$k=5$	$k=9$	pretrained	retrained	
<8	5 147	6.3	24.4	23.8	4.2	44.5
8–16	11 141	0.3	3.3	28.1	18.1	76.6
16–32	7 612	0.0	0.4	24.7	29.5	76.9
32–64	2 226	0.0	0.0	31.8	43.3	77.9
≥ 64	15	0.0	0.0	0.0	0.0	80.0
All	26 141	1.4	6.3	26.6	20.8	70.5

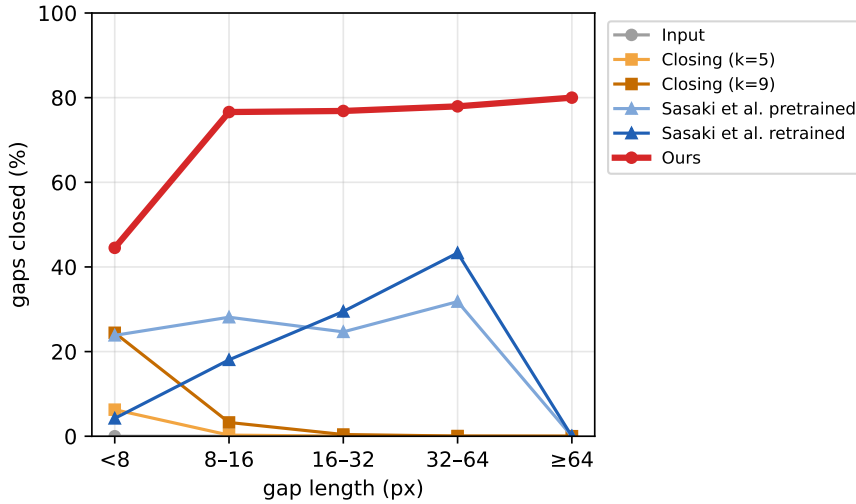


Figure 4: Fraction of erased segments closed as a function of segment length. The last bin (≥ 64 pixels) contains only 15 segments.

with gap length, from 4.2% below 8 pixels to 43.3% between 32 and 64 pixels. Neither variant closes any of the 15 gaps of 64 pixels or more. Our model closes between 76.6% and 80.0% of the gaps in every length bin from 8 pixels upwards, including 77.9% of the 2 226 gaps between 32 and 64 pixels and 12 of the 15 longest gaps, which shows that the long-range context provided by the dilated stack and the two attention modules allows it to bridge gaps much longer than the receptive field of a local filter. Its closure rate is lowest, 44.5%, for the shortest segments (under 8 pixels), although it is still the highest of all methods in that bin. We hypothesize that this is mainly an effect of the closure criterion: for a segment of only a few pixels, a completion that is slightly thinner or offset by one pixel fails to cover 80% of the segment, whereas the same absolute error is negligible for a long segment. Short segments may also lie more often at stroke ends and junctions, where the continuation of the line is more ambiguous. Two observations are consistent with this interpretation: the fill F1 of our model is much higher at $\tau = 2$ than at $\tau = 0$, and the retrained Sasaki et al. network, whose strokes are often offset by a pixel, shows an even sharper drop on short gaps.

Qualitative results. Figure 5 shows crops centered on the largest erased segment of the four preview test images, and Figure 7 shows the same images in full. Our model reconstructs long missing stretches of straight and curved strokes with the correct thickness, continues strokes through junctions, and leaves the rest of the drawing untouched; the few errors (red and blue

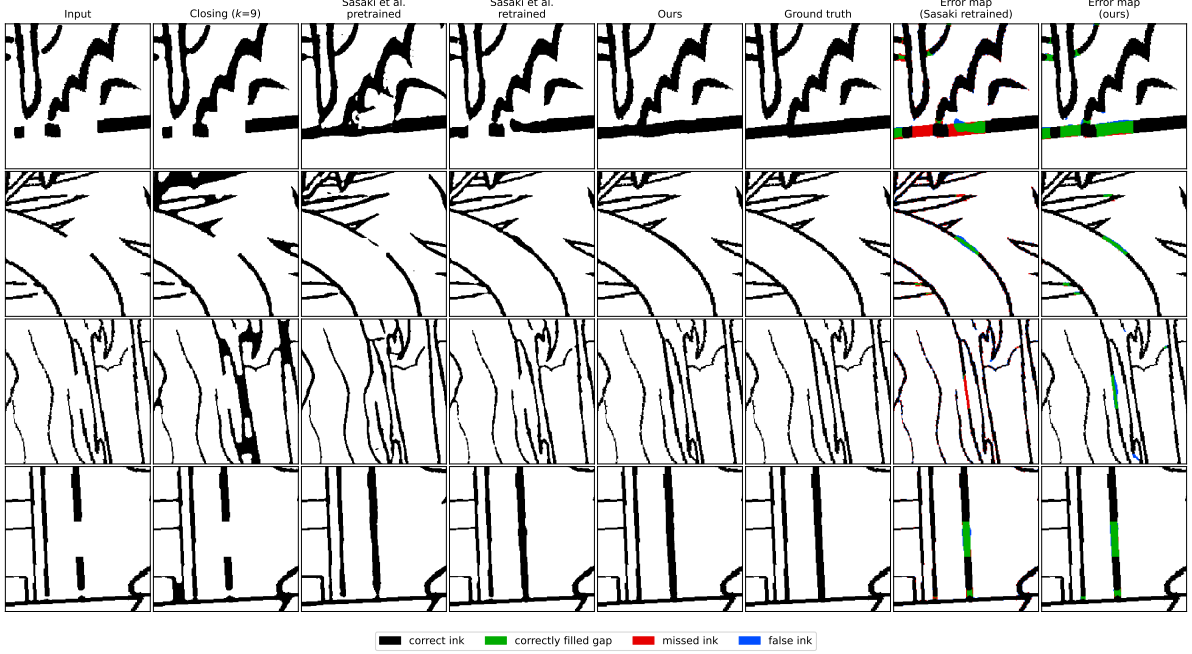


Figure 5: Qualitative results on the four preview test images, cropped to a 224×224 window centered on the largest erased segment. Columns: corrupted input, morphological closing ($k = 9$), the pretrained and retrained Sasaki et al. networks, our binarized output, ground truth, and the error maps of the retrained Sasaki et al. network and of our model (black: correct ink; green: erased ink correctly redrawn; red: missed ink; blue: false ink).

in its error maps) are mostly one-pixel deviations along the boundary of the completed strokes. The pretrained Sasaki et al. network closes several of the gaps but alters the drawing elsewhere, for example by adding spurious strokes in the first row. The retrained network produces strokes of the right thickness, but it leaves part of the long gap of the first row open, and its error maps show small deviations along nearly all existing strokes, where our model reproduces the input exactly. Figure 8 in Appendix A shows the full images for both Sasaki et al. variants.

Failure cases. Figure 6 shows the three test images with the lowest fill F1 among those with at least 200 erased pixels. The dominant failure mode is a long erased stretch of a curved contour whose shape cannot be inferred from its endpoints alone, as in the lobes of the leaf in the first row: the model leaves these gaps open instead of guessing a continuation. The retrained Sasaki et al. network also leaves these same gaps open. A second failure mode appears in regions with several close, nearly parallel strokes or fine hatching (second and third rows), where the model completes some strokes only partially or adds ink slightly off the true stroke. Even in these images, the model does not produce the ink blobs that closing creates.

Computational cost. Our model has 43.4 M parameters. Inference on one 896×896 image takes 91.6 ms in half precision on an NVIDIA Tesla T4 GPU (forward pass only, excluding loading and binarization), with a peak GPU memory use of 1.24 GB, which makes it practical as an interactive preprocessing step before coloring. The Sasaki et al. network is considerably lighter, with 5.5 M parameters and a peak memory use of 0.13 GB; on the same GPU it takes 11.3 ms per image in half precision and 23.0 ms in single precision, in both cases including the transfer of the image to and from the GPU. Because the timings of the two methods were measured slightly differently, the comparison is only approximate, but it indicates that our model is roughly an order of magnitude more expensive, a cost that remains small compared

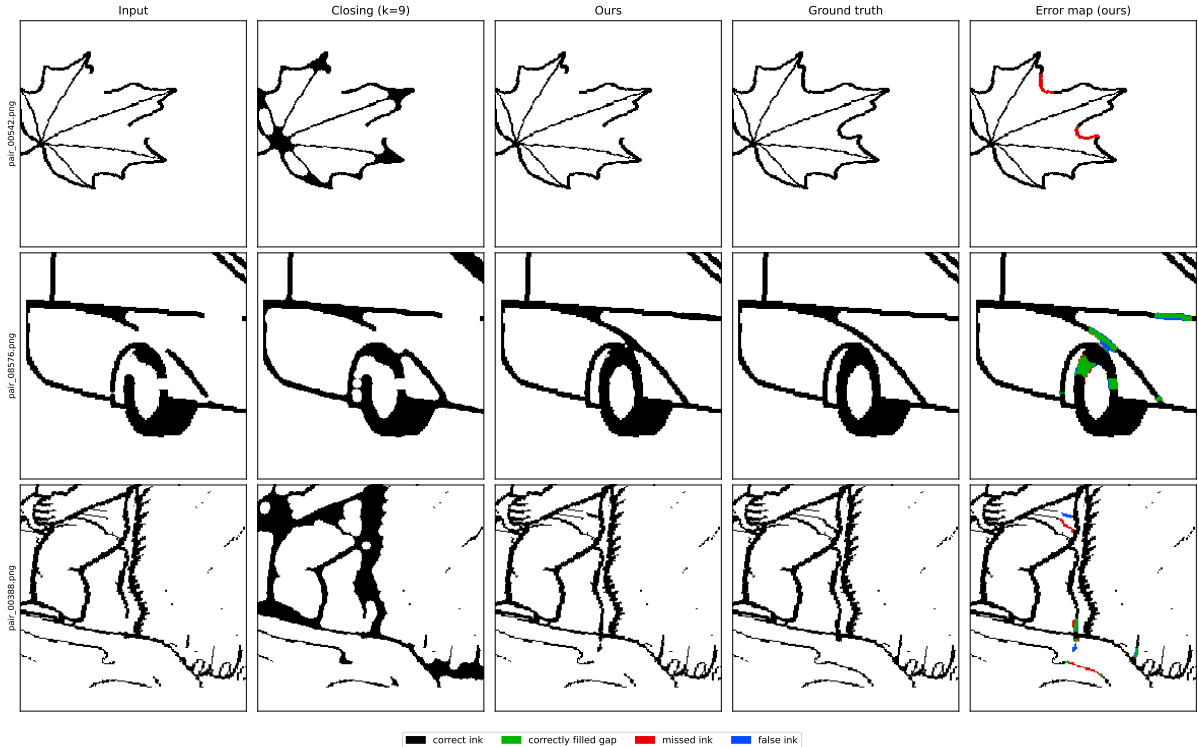


Figure 6: Failure cases: the three test images with the lowest fill F1 of our model among images with at least 200 erased pixels. Columns: corrupted input, morphological closing ($k = 9$), our binarized output, ground truth, and the error map of our output, with the same color code as Figure 5.

with that of manual gap repair.

8 Discussion and Future Work

The results show that the mask-free formulation is learnable with the proposed architecture and loss, and that the resulting model turns drawings in which a third of the regions cannot be filled correctly into drawings in which 94.7% of them can. The comparison with Sasaki et al. [2017] also shows that closing gaps visually is not sufficient for this application: a method that regenerates the whole drawing can recover a large part of the missing ink and still produce more leaks than the unrestored input, because any stroke it weakens elsewhere becomes a new gap. Preserving the input exactly outside the gaps is as important as filling them. Several points limit the current conclusions and set the direction for further work.

Remaining leaks. Although the model separates 94.0% of the merged regions, a single remaining gap is enough to spoil a bucket fill, and 55.0% of the restored images still contain at least one leak; region count accuracy (38.0%) reflects the same effect. From the point of view of a user, however, the remaining work is reduced from about 15 corrections per image to one or two in most cases. Weighting the loss towards gaps that actually merge regions, rather than towards all hole pixels equally, and improving the closure of very short gaps are two direct ways to increase the fraction of leak-free drawings.

Baselines and ablations. The retrained Sasaki et al. network follows the original training recipe, so its difference from our model combines the effects of architecture, loss, and optimizer. Training the same architecture with our loss would separate the contribution of the architecture

from that of the loss, and ablations of the individual components of our model (gated convolutions, dilated stack, each attention module, and each loss term) would further attribute the improvement. Other comparisons are also left for future work: gap-tolerant region segmentation such as trapped-ball filling [Zhang et al., 2009], endpoint-extension heuristics, and mask-guided inpainting methods run with an oracle hole mask as an upper-bound reference.

Synthetic corruption. All numbers are obtained with synthetic corruption applied to already-clean line art (Section 4). This lets us construct paired data at scale and compute exact hole masks for training and evaluation, but it leaves open the question of how well the restoration transfers to gaps produced by real production processes, such as scanning noise, lossy compression, or line thinning, which may differ in shape and scale from the synthetic corruption. The modest zero-shot results of the pretrained Sasaki et al. network on our data illustrate how strongly learned gap-closing models can depend on their training distribution. Validating the approach on real, naturally occurring gaps is planned future work.

Metrics. Region count accuracy is sensitive to any single remaining error in an image, which makes it a conservative metric, and the bucket and leak metrics treat all regions equally regardless of their size. A metric that reflects the practical cost of a remaining gap more closely, for example by weighting leaks by the area of the regions they affect, may be more informative for this application.

Scope. The current model operates on a single fixed resolution (896×896) and on binary, anti-aliased line art. Generalization to other resolutions, line styles, and stroke widths has not been evaluated.

9 Conclusion

We formulated the restoration of broken lines in binary line art, for the purpose of enabling flood-fill coloring, as a mask-free task in which the network must locate and close gaps without access to a hole mask at inference time. We described a network combining gated convolutions, a dilated context stack, and two-scale self-attention, a loss that combines hole-weighted cross-entropy with global and gap-local Dice terms, and evaluation metrics centered on gap closure and simulated paint-bucket behavior rather than on pixel accuracy. On a held-out test set of 1 000 images, the model closes 70.5% of the erased segments, with a closure rate that stays above 76% for gaps from 8 to more than 64 pixels long, raises the fraction of correctly paintable regions from 64.1% to 94.7%, reduces leaks by 90.5%, and makes 45.0% of the drawings entirely leak-free, while preserving the ink already present in the input. Neither morphological closing nor the joint gap detection and inpainting network of Sasaki et al. [2017], even when retrained on the same data, closes more than 26.6% of the gaps or improves on the bucket accuracy of the unrestored input. Ablating the components of our model, comparing against further baselines, and evaluating on naturally occurring rather than synthetic gaps are the next steps of this work.

References

- Evan Casey, Víctor Pérez, and Zhuoru Li. The animation transformer: Visual correspondence via segment matching. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- Ke Chen, Irina Rabkina, Matthew McLure, and Kenneth D. Forbus. Human-like sketch object recognition via analogical learning. In *Proceedings of the AAAI Conference on Artificial*

- Intelligence*, volume 33, pages 2613–2620, 2019. doi: 10.1609/aaai.v33i01.33012613. URL <https://www.qrg.northwestern.edu/Resources/CBO/>.
- Yuekun Dai, Qinyue Li, Shangchen Zhou, Yihang Luo, Chongyi Li, and Chen Change Loy. Paint bucket colorization using anime character color design sheets. *arXiv preprint arXiv:2410.19424*, 2024a.
- Yuekun Dai, Shangchen Zhou, Qinyue Li, Chongyi Li, and Chen Change Loy. Learning inclusion matching for animation paint bucket colorization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 25544–25553, 2024b.
- Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, New York, 1994.
- Sébastien Fourey, David Tschumperlé, and David Revoy. A fast and efficient semi-guided algorithm for flat coloring line-arts. In *Vision, Modeling and Visualization (VMV)*. The Eurographics Association, 2018. doi: 10.2312/vmv.20181247.
- Xiaoling Hu, Fuxin Li, Dimitris Samaras, and Chao Chen. Topology-preserving deep image segmentation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Masahiro Kono, Akinobu Maejima, Yuki Koyama, Yotam Sehayk, and Takeo Igarashi. No pixel left behind: Filling gaps in anime colorization. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI)*, 2026.
- Hangyu Lin, Yanwei Fu, Xiangyang Xue, and Yu-Gang Jiang. Sketch-BERT: Learning sketch bidirectional encoder representation from transformers by self-supervised learning of sketch gestalt. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- Guilin Liu, Fitsum A. Reda, Kevin J. Shih, Ting-Chun Wang, Andrew Tao, and Bryan Catanzaro. Image inpainting for irregular holes using partial convolutions. In *European Conference on Computer Vision (ECCV)*, pages 85–100, 2018.
- Xueting Liu, Tien-Tsin Wong, and Pheng-Ann Heng. Closure-aware sketch simplification. *ACM Transactions on Graphics*, 34(6), 2015.
- Zhiheng Liu, Ka Leong Cheng, Xi Chen, et al. MangaNinja: Line art colorization with precise reference following. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi. V-Net: Fully convolutional neural networks for volumetric medical image segmentation. In *International Conference on 3D Vision (3DV)*, pages 565–571, 2016.
- Haoran Mo, Chengying Gao, and Ruomei Wang. Joint stroke tracing and correspondence for 2D animation. *ACM Transactions on Graphics*, 43(3), 2024. doi: 10.1145/3649890.
- Amal Dev Parakkat, Pooran Memari, and Marie-Paule Cani. Delaunay painting: Perceptual image colouring from raster contours with gaps. *Computer Graphics Forum*, 41(6):166–181, 2022.

- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.
- Ruben Tous. InkCloser Dataset. Dataset, 2026. URL https://drive.google.com/file/d/1N08i_J5mSlh7YYuwH2ZyFqkIW7ISrGuk/view?usp=drive_link. Accessed: 2026-10-06.
- Kazuma Sasaki, Satoshi Iizuka, Edgar Simo-Serra, and Hiroshi Ishikawa. Joint gap detection and inpainting of line drawings. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5725–5733, 2017.
- Kazuma Sasaki, Satoshi Iizuka, Edgar Simo-Serra, and Hiroshi Ishikawa. Learning to restore deteriorated line drawing. *The Visual Computer*, 34(6–8):1077–1085, 2018.
- Jean Serra. *Image Analysis and Mathematical Morphology*. Academic Press, London, 1982.
- Suprosanna Shit, Johannes C. Paetzold, Anjany Sekuboyina, et al. clDice – a novel topology-preserving loss function for tubular structure segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16560–16569, 2021.
- Edgar Simo-Serra, Satoshi Iizuka, and Hiroshi Ishikawa. Mastering sketching: Adversarial augmentation for structured prediction. *ACM Transactions on Graphics*, 37(1), 2018.
- Li Siyao, Tianpei Gu, Weiye Xiao, Henghui Ding, Ziwei Liu, and Chen Change Loy. Deep geometrized cartoon line inbetweening. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Pierre Soille. *Morphological Image Analysis: Principles and Applications*. Springer, Berlin, 2nd edition, 2003.
- Roman Suvorov, Elizaveta Logacheva, Anton Mashikhin, et al. Resolution-robust large mask inpainting with Fourier convolutions. In *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2149–2159, 2022.
- Daniel Šýkora, John Dingliana, and Steven Collins. LazyBrush: Flexible painting tool for hand-drawn cartoons. *Computer Graphics Forum*, 28(2):599–608, 2009.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Yi Wang, Ying-Cong Chen, Xin Tao, and Jiaya Jia. VCNet: A robust approach to blind image inpainting. In *European Conference on Computer Vision (ECCV)*, 2020.
- Yuxin Wu and Kaiming He. Group normalization. In *European Conference on Computer Vision (ECCV)*, pages 3–19, 2018.
- Jiahui Yu, Zhe Lin, Jimei Yang, Xiaohui Shen, Xin Lu, and Thomas S. Huang. Free-form image inpainting with gated convolution. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4471–4480, 2019.
- Matthew D. Zeiler. ADADELTA: An adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.
- Song-Hai Zhang, Tao Chen, Yi-Fei Zhang, Shi-Min Hu, and Ralph R. Martin. Vectorizing cartoon animations. *IEEE Transactions on Visualization and Computer Graphics*, 15(4): 618–629, 2009.

A Complete Results

Table 4 reports every metric of Section 6 for every method (point estimates). Figure 7 shows the full preview test images corresponding to the crops of Figure 5 for our model, and Figure 8 shows them for both Sasaki et al. variants.

Table 4: All metrics on the test set (1 000 images). Percentages unless noted.

Metric	Input	Closing		Sasaki et al.		Ours
		$k=5$	$k=9$	pretrained	retrained	
Fill P ($\tau=0$) $\uparrow$	–	1.1	1.2	10.7	28.1	73.6
Fill R ($\tau=0$) $\uparrow$	0.0	1.8	6.3	54.0	56.5	84.0
Fill F1 ($\tau=0$) $\uparrow$	–	1.4	2.1	17.8	37.6	78.5
Fill P ($\tau=2$) $\uparrow$	–	100.0	91.3	85.1	97.6	91.7
Fill R ($\tau=2$) $\uparrow$	0.0	6.1	14.6	78.9	78.9	93.9
Fill F1 ($\tau=2$) $\uparrow$	–	11.6	25.2	81.9	87.2	92.8
Pixel IoU $\uparrow$	96.8	92.1	83.7	82.0	88.8	98.6
Pixel F1 $\uparrow$	98.4	95.9	91.1	90.1	94.1	99.3
Ink kept $\uparrow$	100.0	100.0	100.0	95.1	94.1	100.0
Ink ratio $\rightarrow 1$	0.968	1.021	1.129	1.081	0.974	1.004
Gap closure $\uparrow$	0.0	1.4	6.3	26.6	20.8	70.5
Region count acc. $\uparrow$	0.1	1.4	2.8	5.5	9.2	38.0
Region recovery $\uparrow$	0.0	36.7	38.6	60.6	33.7	94.0
Bucket acc. (IoU ≥ 0.9) $\uparrow$	64.1	37.6	22.8	26.2	36.4	94.7
Mean region IoU $\uparrow$	70.9	58.3	45.1	57.4	56.6	96.5
Leaks / image $\downarrow$	15.46	13.08	9.45	13.77	19.36	1.47
Leak-free images $\uparrow$	0.1	0.2	0.3	4.2	8.5	45.0

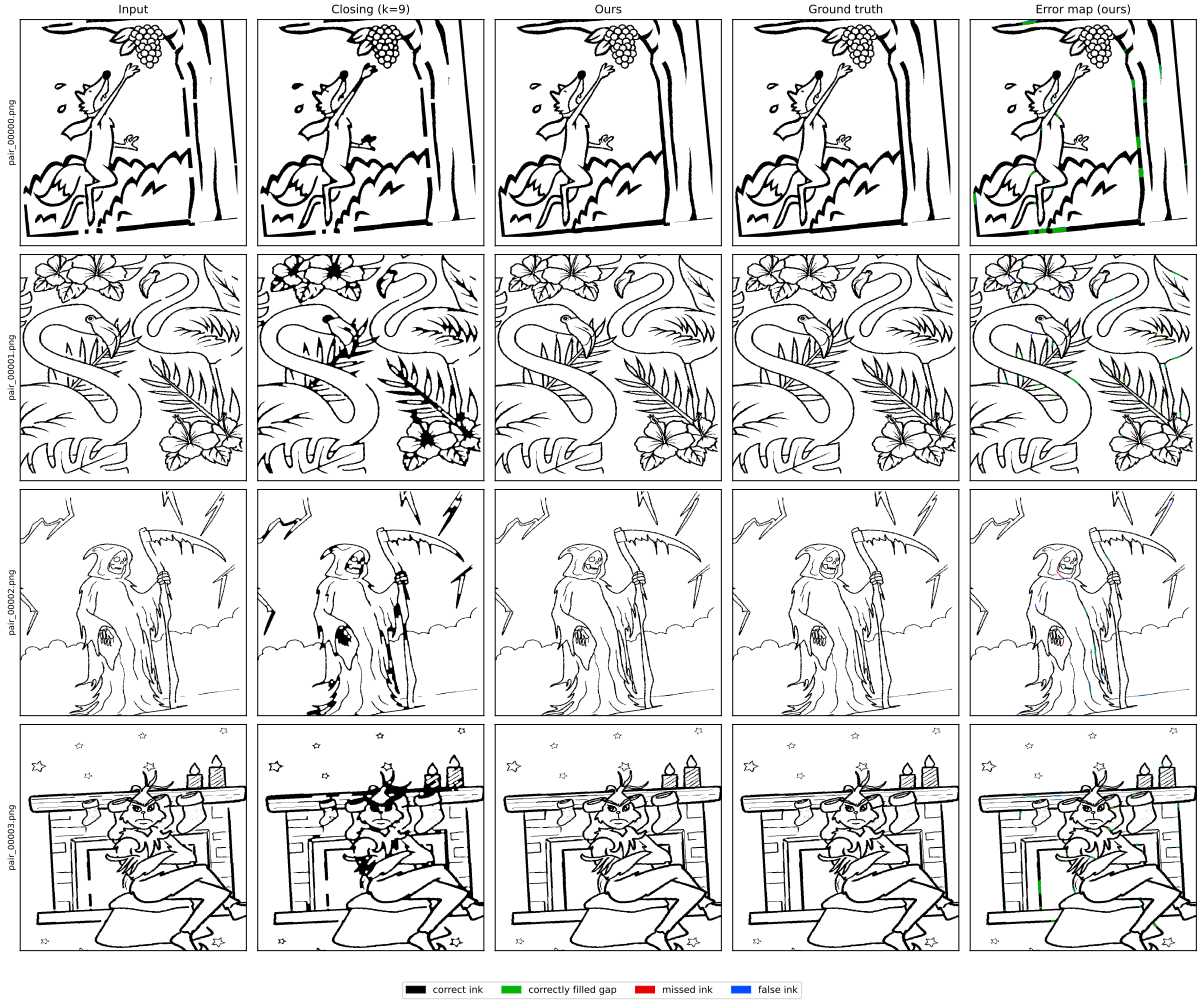


Figure 7: Full 896×896 preview test images: corrupted input, morphological closing ($k = 9$), our binarized output, ground truth, and the error map of our output. Same color code as Figure 5.

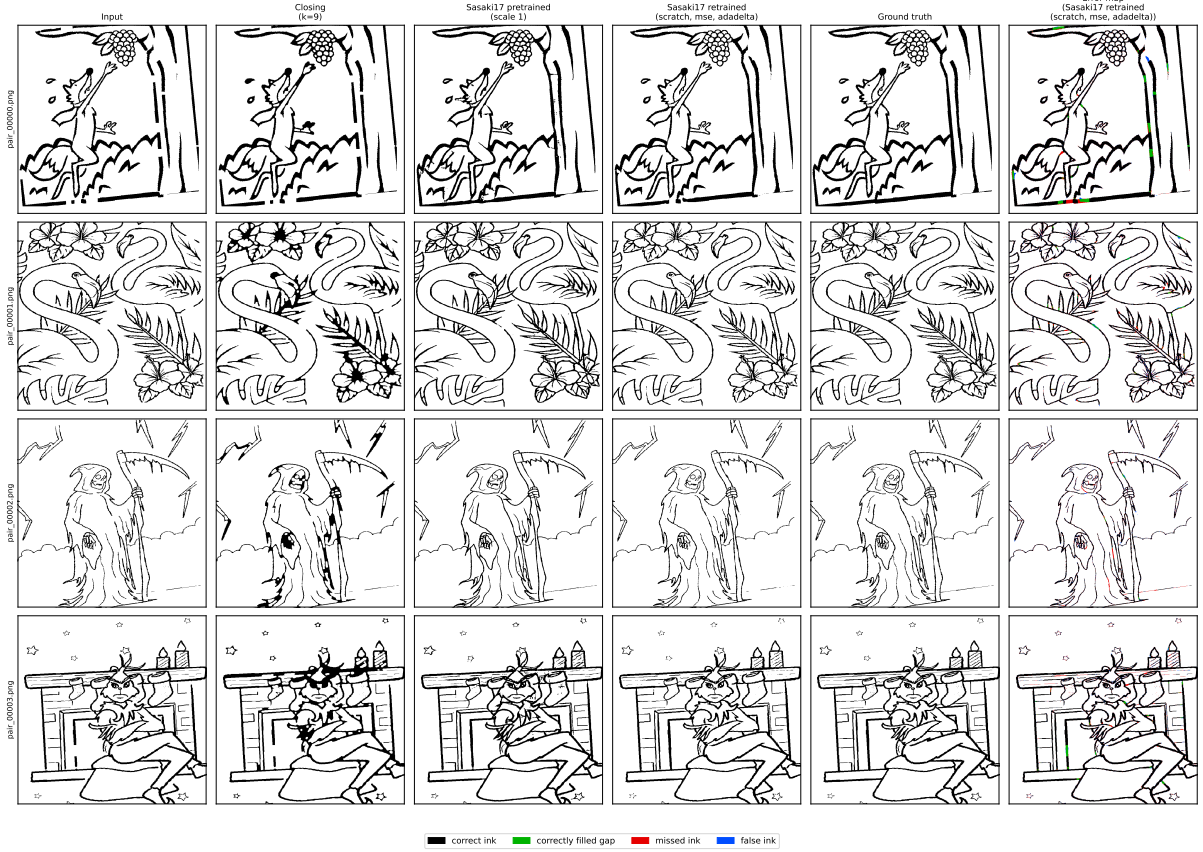


Figure 8: Full 896×896 preview test images for the Sasaki et al. network: corrupted input, morphological closing ($k = 9$), the pretrained and retrained variants, ground truth, and the error map of the retrained variant. Same color code as Figure 5.